

This work can be cited as:

Ahmad, M. O., Kuvaja, P., Markkula, J. & Oivo, M. (2016). Transition of software maintenance teams from Scrum to Kanban. IEEE 49th Hawaii International Conference on System Sciences (HICSS).

Transition of Software Maintenance Teams from Scrum to Kanban

Muhammad Ovais Ahmad, Pasi Kuvaja, Markku Oivo, Jouni Markkula
M-Group, Faculty of Information Technology and Electrical Engineering, University of Oulu, Finland
{Muhammad.Ahmad; Pasi.Kuvaja; Markku.Oivo; Jouni.Markkula}@oulu.fi

Abstract

Software companies are increasingly moving to use Kanban after Scrum since Kanban claimed to offer improved project visibility, software quality, team motivation, communication and collaboration. However, empirical studies are needed to verify these effects and companies' real motivation for the transition. In particular, underlying reasons leading software companies to start using Kanban in software maintenance must be understood. A multiple-case study was conducted to investigate why two experienced Scrum maintenance teams transitioned to Kanban. We conducted 17 semi-structured interviews with two different teams from two large Finnish software companies. Thematic analysis was applied to identify the most common challenges in Scrum and their solutions in Kanban. The results showed that Scrum maintenance teams faced challenges, such as lack of work visibility, task prioritisation, communication and collaboration, over commitment of sprints, work synchronisation and changing people. We discuss how maintenance teams mitigated these challenges with Kanban and present the lessons learned.

1. Introduction

The IEEE defined software maintenance as ‘the process of modifying a software system or component after delivery to correct faults, improve performance or other attributes, or adapt to a changed environment’ [3]. Software maintenance activities are often performed by using a heavyweight process or even without any specified process [4]. Various maintenance models, such as quick-fix, Boehm, Osborne, iterative-enhancement, full reuse and the IEEE-1219 and ISO/IEC 14764 IEEE

standards for maintenance have been reported [6, 7]. The challenge faced in existing software maintenance models is that they largely derive from the waterfall life cycle model. They have limitations when handling problems, such as unstructured code, team morale, poor visibility, lack of communication between stakeholders and frequent customer request prioritisation [4]. These limitations can be avoided by using Agile methodologies, which focus on customer collaboration, continuous testing, refactoring and incremental development [11, 12]. Agile methods, including Kanban, have become popular for software development [1, 5, 19, 21]. Kanban aims to provide visibility and improve communication and collaboration throughout the development and maintenance processes [19, 20, 21, 22].

Thus far, an Agile method for maintenance has not been studied in detail [11, 16, 17, 18]. Nonetheless, some pioneering studies have suggested that agile practices have several advantages in software maintenance [8, 11, 15, 16]. Cockburn [14] argued that Agile methods subsumed maintenance activities in development owing to their iterative sequence of software deliveries. However, there is no evidence that this has become common practice [11].

We conducted a multiple-case study of two maintenance teams in two large companies that have changed their work practices from Scrum to Kanban. We conducted this study under the Need for Speed (N4S) research program (www.n4s.fi), which is a large Finnish national program aiming to create the foundation for Finnish software-intensive businesses in the new digital economy. In the study we conducted a set of semi-structured interviews for collecting data to address the following research questions:

RQ1: What are the challenges in using Scrum in maintenance teams?

RQ2: How can Kanban provide solutions to tackle the challenges in Scrum maintenance teams?

The remainder of this paper is organised as follows: In Section 2, we provide information on related works and outline the differences between Scrum and Kanban. Sections 3 and 4 present our research method, data collection and analysis and results. Sections 5, 6 and 7 present the validity discussion and lessons learned. Section 8 concludes the paper with future research directions.

2. Background and related works

Bennett and Rajlich [25] defined software maintenance as ‘*the modification of a software product after delivery*’. During maintenance, software moves from its original state [24] into continuous evolution [29]. According to the classic definition by Lientz and Swanson [26], maintenance consists of four types: (i) adaptive (responding to changes in the software environment), (ii) perfective (responding to new user requirements), (iii) corrective (fixing errors) and (iv) preventive (preventing foreseeable problems in the future).

Approximately 75% of maintenance is adaptive or perfective and is thus effectively new development [11]. The adaptive and perfective nature of maintenance serves as a good argument for adopting Agile methods. Software development and maintenance works are different in nature. For example, software development projects usually deal with one instance of a system at a time. On the other hand, maintenance involves dealing with multiple customers with different customised versions of a system [11, 25]. This results in many small independent tasks with limited cohesion rather than in the naturally homogeneous tasks in development projects.

2.1 Agile methods in software maintenance

Agile methods include common characteristics, such as an iterative development process, face-to-face communication, light documentation, frequent testing, knowledge transfer through openness and a focus on a high quality of code and product [1, 27]. Studies [9, 16, 19, 27] have reported that Agile methods can help to speed up the process and improve code quality, user satisfaction, team motivation and communication between members of the maintenance team. Additionally, some maintenance teams successfully implement daily Scrum meetings [15], and others easily adopt pair programming; both improve code quality [12, 24].

Kanban entered into software development in 2004, when David Anderson introduced it in practice while assisting a software development team at Microsoft [30]. The motivation to use Kanban was to visualise

work, limit work in progress (WIP) and identify process constraints to achieve flow and focus on a single item at a given time [5, 20]. Kanban has five core principles: visualise workflow, limit work in progress, measure and manage flow, make process policies explicit and use models to recognise improvement and opportunities [30]. Kanban principles are applied using a Kanban board, which visualises the flow of activities of the process in various columns. Cards are used for each working item on the Kanban board to show its current state. The flow of work items through the process is optimised by limiting the WIP in each activity column to a maximum number of items that can be pulled into the column. In this manner, the team visualises their workflow, limits WIP items in each stage and effectively monitors the cycle time from start to finish. Sjøberg, Johnsen, and Solberg [19] compared the usage of Scrum and Kanban in a medium-sized firm across a period of two years. Their findings show that with Kanban the number of weighted bugs and bugs decreased by 10% and 11%, respectively, and productivity improved by 21% for implemented features. In general, Kanban focuses on communication, collaboration and integration between software developers, testers and support teams, resulting in rapid software development and continuous delivery to the customer. Additionally, Kanban in software development brings visibility to work and improves work efficiency and throughput [4, 19, 27, 30, 31]. Industrial practices have shown that Kanban can reduce maintenance and development costs by limiting the WIP items [4, 19, 31]. A simulation study shows that WIP limits can improve maintenance throughput and efficiency and reduce cost [4].

Some studies [12, 27, 29] have reported that Agile methods are very difficult to implement in the maintenance area. For example, maintenance work can be organised into sprints; however, maintenance typically does not involve task synergy and an assumed common goal as in the case of development work. Maintenance teams typically work with many customers who are seldom on-site, and there is often little close interaction. Close cooperation with the customer is regarded positively in agile teams; however, there are no documented examples of maintenance teams with on-site customers in the scientific literature. Further, openness is highly valued in maintenance [11, 12, 24].

2.2 Differences between Scrum and Kanban

Both Scrum and Kanban belong to Agile and Lean methods, where the focus is to quickly respond to customer requests. Both are highly adaptive and based on highly collaborative and self-managing teams. Kniberg and Skarin [2] explained that Scrum is more prescriptive than Kanban. Scrum gives you more

constraints and thereby leaves fewer options open, e.g. Scrum prescribes the use of timeboxed iterations, Kanban doesn't. Further, Kniberg and Skarin [2] briefly presented the differences between Scrum and Kanban; these are summarised in Table 1.

Table 1. Differences between Scrum and Kanban

Kanban	Scrum
No prescribed roles	Predefined roles of Scrum master and team member
Continuous delivery	Timeboxed sprints
Work is 'pulled' through the system (single piece flow)	Work is 'pulled' through the system in batches (sprint backlog)
Changes can be made at any time	No changes allowed in mid-sprint
Uses lead time as default metric for planning and process improvement	Uses velocity as default metric for planning and process improvement.
Appropriate in operational environments with a high degree of variability in priority	Appropriate in situations where work can be prioritised in batches that can be left alone
Commitment optional	Team commits to a specific amount of work for this iteration
WIP limits directly (per workflow state)	WIP limits indirectly (per sprint)
A Kanban board is persistent	A Scrum board is reset between each sprint

There is a difference between Scrum and Kanban boards. Scrum has two modes: (i) plan mode and (ii) work mode. The Scrum board is used by teams to plan work before it actually starts. In the plan mode, teams divide their work in sprints and assign story points to user stories to help plan which story goes to which sprint. In this case, a list of items, called a backlog, is created, a number of items are moved to sprints, and a time estimate is given to each item. The work mode is the actual board itself where you can move cards across status columns. In contrast, Kanban does not have a plan mode. Kanban uses the same column-based interface as Scrum for tracking the status of items without organising work into sprints.

3. Research method

The research presented in this paper is a multiple-case study [13] of two software maintenance teams who changed their work practices from Scrum to Kanban. We chose a multiple-case study to provide multiple sources of evidence that would better validate the findings [13]. We used purposeful sampling [13] and looked for software maintenance teams that had experience in using Kanban in their work. The benefit of purposeful sampling is to allow the researcher to get an understanding of the phenomenon from the selected experienced participants. We chose participants from two large Finnish software companies who had

knowledge on the topic, were involved in Lean and Agile projects and had been using Kanban for more than one year. These large multinational companies are transforming their way of working based on Agile and Lean approaches. Introducing Kanban to various teams in their companies is part of that transformation.

Company contact persons identified candidate teams from each company. Both maintenance teams consist of 6–8 team members with two managers. The teams' services portfolio consists of customers' requests, which come from around the world. The members in both maintenance teams had more than ten years of experience, while two members were new in software maintenance. All of the team members had practical experience with Agile methods.

We collected data from two teams in December 2014, using semi-structured interviews [28]. The interview questions were based on three main themes: (i) participant background information, (ii) what challenges in Scrum motivated teams to start using Kanban and (iii) how teams overcome the previously faced challenges using Kanban. The purpose of the research was explained to the participants before we started the interviews. From each team, we interviewed product owners, project managers, coaches, Scrum masters, team leaders, developers, testers and trouble reporters. Altogether, we performed 17 interviews (9 face-to-face interviews and 8 via Skype), each lasting 1–2 h. Table 2 provides a summary of the participants.

Table 2. Summary of study participants

Interviewee's title	No. of interviewees
Product owner	2
Agile, lean coach	3
Project and line manager	3
Scrum master, team leader	3
Software developer/software tester/trouble reporter	6
Total	17

One researcher who had participated in the interviews checked each transcription by listening to the tape and simultaneously correcting possible transcription errors. The interview transcripts were analysed using Nvivo 10 software with thematic analysis. Thematic analysis is a way of seeing themes and patterns from the collected information [10]. Template analysis was chosen because it allows a priori themes to be used to help and develop initial versions of the coding template. An initial interview template was constructed that focused on various aspects of maintenance team work: work visibility, task prioritisation and assignment and team communication and coordination. Interview data were then mapped onto the initial template and modified until all relevant data were coded. The process was concluded by applying the final version of the template to the data as a whole, and a priori themes were redefined or

discarded if they did not prove to be helpful in capturing key details from the data.

4. Results

Teams in this study came from two large global software companies that have branches in European countries, USA, and Asia. Until 2013, both maintenance teams were using Scrum. The teams used Scrum with the following elements:

- Two-week sprints
- Daily stand-up meetings
- Planning meetings before each sprint
- The Scrum master role was assigned to one member of the maintenance team
- The product owner role was assigned to several internal team members owing to multiple customer relations and organisational structure
- All the maintenance tasks and customer requests were collected into a prioritised product backlog
- Based on the highest priority, a number of tasks were selected for each sprint.

Both teams realised that the sprints required by the Scrum method did not fit into the hectic software maintenance process. The customer was not on-site, teams selected release dates almost randomly and unpredicted tasks that needed immediate attention came from customers in the middle of a sprint. When a customer escalated a severe problem, the concepts of sprints and sprint commits were not possible. Additionally, daily stand-up meetings lasted longer than planned and took more than 30 min, and as a result, team members were not motivated to attend. In a nutshell, our initial analysis identified six interconnected challenges with both maintenance teams using Scrum. Table 3 summarises these challenges and their interview coding frequency. Similar challenges in using agile methods are also faced by Software as a Service and Mass-Produced Embedded Systems development teams [23, 32].

Table 3. Explanation of challenges

No.	Challenges	Explanation
1	Lack of work visibility	Teams working on multiple customers' requests. Members are unaware of each other's tasks. Overall task status is missing, especially for incoming and blocked tasks.
2	Fluctuating task priorities	Sprints make it difficult to handle daily task priorities. The teams have to abandon much of their current work to deal with the emergency or high-priority tasks.
3	Unpredictable tasks frequently in process	Maintenance teams are tackling vastly different, unpredictable and critical tasks that require quick responses. Such tasks are difficult to handle in the sprint.
4	Team sprint commitments	Maintenance teams are unable to forecast what they would be working on for the next two weeks. This makes sprint commitment difficult.
5	Work synchronisation and changing people	Lack of work synchronisation is an indication that coordination among management, development and maintenance teams is neither frequent nor effective.
6	Lack of communication and collaboration	Teams were not conducting proper stand-up meetings, and team members were mostly not involved in critical decisions.

4.1 Challenge 1: Lack of work visibility

Lack of work visibility is one of the biggest issues for both maintenance teams. Task visibility is not only important for successful completion of tasks but also needed by management to see the right level of information at the right time. Owing to the lack of visibility, both teams faced three main problems: (1) It is difficult to see what the other teams are doing because at any given time, too many tasks are running in parallel, which makes it difficult to know who is doing what task; (2) a lot of task functionalities were not in sync because dependences from other teams were not visible; and (3) there is not enough time for each member to go through all tasks and ask for information about their assigned task inside and outside the team. A product manager and trouble reporter explained this

challenge as *'The main problems were lack of visibility for the whole project (Product Managers). It was like everybody was doing and you actually need to go and ask, what you are doing (Trouble Reporter). Essentially [there is a] lack of visibility in terms of the work in progress. So as we didn't have enough visibility what work is going on, in the different phases'*.

4.2 Challenge 2: Fluctuating task priorities

Task prioritisation is a very important activity that needs to be performed carefully to deliver value to customers and reduce the risk of costly software failures. The interviewees reported that lack of clear decisions and constant reprioritisation of tasks made it difficult to complete the work on time and made it challenging to find a balance between development and maintenance work. The continuous change in tasks priorities during sprints is extremely challenging to handle. The teams have to abandon much of their current work to deal with emergency or high-priority tasks. The interviewees expressed task prioritisation challenges as follows: *'We have our priorities defined, but we have to deal with change on a regular basis so Scrum wasn't working that well, in a sense of our needs. Project is more unexpected, we change more the tasks. The priorities start changing all the time. One day, we need to do very urgent tasks and next day that are not urgent; the things change a lot (Developer). The challenges are always the same, trying to respect the priorities, trying to get all the team involved in the most important tasks and try not to start too many subjects at the same time. Since we are always switching our priorities, maybe we can change our way of work (Developer)'*.

4.2 Challenge 3: Unpredictable tasks frequently in process

Maintenance teams are tackling a vast variety of unpredictable and critical tasks, for example, work on isolated or short-timeframe tasks that require quick responses. Such tasks are difficult to handle in a sprint. In Scrum, sprints allow the team to take a specific number of tasks and deliver minimum viable products at the end of the sprint. The sprint-based Scrum way of working was too rigid for both maintenance teams in our study. Sprints create too much overhead, which started to cause burnout for teams. Most of the sprint commitments were not achieved because of daily frequent changes in tasks and dependencies on other teams. As interviewees mentioned, *'things really change daily. It is tough to react when we had a mindset of Scrum [two weeks sprint] and protecting the teams from changes during sprints (Project Manager). We get a lot of dependencies from a lot of other projects. There's no real possibility of not changing stuff during sprints. I think Scrum doesn't work well for maintenance (Line Manager). A strict sprint period*

[two-week sprint], it's not enough to get something to a clear end that we can demonstrate (Coach)'. Additionally, sometimes the team completed the sprint task, but they were still waiting for the sprint to end to be able to present the work in a demo. A coach expressed this as follows: *'if you took one and a half sprints, why then wait for a demo and get onwards to this sprint if you had something ready [in between], so why not do it more in a continuous thing'*.

4.4 Challenge 4: Team sprint commitments

In sprints, a team commits to getting a certain amount of work done at the end. Scrum protects the team from unexpected work in each sprint and delivers the committed work. Such commitments are critical because without commitments, the timebox fails, velocity metrics fail and the team starts to lose trust. In the case teams, the product owner brought in high-priority tasks in meetings from which teams made commitments for the whole sprint at the beginning. The teams are unable to forecast what they would be working on for the next two weeks. Additionally, in the middle of a sprint, team members accept more tasks as they come from higher management. In this manner, teams start working on too many tasks at the same time, which makes it difficult to achieve sprint commitments. This challenge is expressed by the interviewees as follows: *'We start a lot of tasks but we never finish those tasks (Developer). We were keeping the commitment to that plan and it was difficult to change it (Tester)'*. Additionally, at any given time, each team member works on multiple tasks owing to business pressures. Such unstable and unpredictable task flow brings complexity and makes it difficult to achieve sprint commitments. This challenge is expressed by the interviewees as follows: *'When you think that you are working with multiple parallel activities, you actually are not working with all of those, they are just hanging there; but you just give an impression that, yes, these are ongoing even though they are not. Most of our sprint commitments never succeeded (Coach). Scrum wasn't responding to our needs (Developer)'*.

4.5 Challenge 5: Work synchronisation and changing people

The management needs up-to-date information about the progress of any project, which is important for resource allocation and task distribution. In both case teams, the lack of work synchronisation with management makes it very difficult to change people from one project or team to another. The lack of work synchronisation is also a sign of poor coordination inside and outside teams as well as with management. This was expressed by our interviewees in this manner: *'There was no synchronisation between the management team and the R&D teams, sometimes they were not aligned; sometimes because of the*

management team, sometimes because of the R&D teams (Developer). It is almost impossible or hard to switch people into different projects (Coach)’.

4.6 Challenge 6: Lack of communication and collaboration

Communication is a challenge in agile environments, especially when multiple teams are working in parallel on a project. Daily stand-up meetings are meant to foster frequent communication and provide up-to-date information about team members and project work. It becomes problematic when teams have poorly conducted daily stand-up meetings, as with our case teams. The lack of proper stand-up meetings broke the communication and collaboration in teams, which made teams work in isolation. Additionally, some team members were not present at stand-up meetings from the beginning or were not paying attention to what others presented as they considered it irrelevant to them. An interviewee expressed this challenge as follows: *‘Teams work more by themselves, each one of them has their own backlog and what to do; the main idea was that they separately did their work and, at the end of the sprint, it is presented to the team (Tester)’*. Additionally, lack of communication made the team members develop a mindset of completing only their own tasks. With such a mindset, a team is not aware of what nearby teams or members are doing. This challenge was reported by interviewees as follows: *‘I finished my part so I can forget about it and that’s it (Product Manager). You might not even know what others are doing (Coach). Everyone just knew about this, for me everything is okay, for my team as well and in the end, we were in trouble. No one really has the overall picture and the dependencies from other projects (Line Manager)’*.

5. Intervention of Kanban

Since 2013, both companies have been transforming their processes from basic Agile principles to Lean principles. In 2013, the case companies’ R&D departments, change management departments and maintenance teams realised that using Scrum practices needed attention owing to the challenges mentioned in Table 3. The management decided to introduce Kanban to maintenance teams.

The companies provided a minimum of one week of training for their maintenance teams through external Kanban consultants and internal coaches. The Kanban intervention strategy was organised in collaboration with the maintenance team managers and change management group at the company level. It was decided that Kanban would first be used in the teams on an experimental basis. After the experimental trial period, the teams and managers decided to continue with the Kanban way of working. Kanban provided opportunity

to interact with people more openly compared to Scrum.

5.1 Solution to lack of work visibility

Scrum and Kanban both visualise the work; the difference is the Scrum board resets between each iteration while the Kanban board is normally persistent and doesn’t need to reset and start over. Further, tasks are visualised on the Scrum board for each sprint, while Kanban visualises tasks that can be pulled at any time to respect WIP limits; this restricts the allowed number of tasks in every workflow state [2]. The goal of practicing Kanban is to visualise and improve the flow of value by optimising a cycle time while respecting WIP limits.

When Kanban was introduced to both maintenance teams, they noticed that there was too much work in progress, and less was completed as well as work became visible to the management and teams. A coach and line manager shared their experience as follows: *‘The great benefit is visibility and traceability... [Kanban] will show the process end-to-end. We have a better vision on where we are, where we stand, what needs to be done, on a daily basis, keeping track of everyday evolution of the several tasks that we have. We see all the cases that we have to work with. Every team sees these cases, in site A and site B. We could track what kind of cases they work with and who they are working with at the moment’*.

The maintenance team members became more aware of their individual tasks as well as those inside and outside their team. Their tasks priorities became clearer owing to the proactivity of individuals and team leaders. The interviewees explained that Kanban helps in the *‘visualisation of your own tasks and to be able to organise yourself a bit more and not lose track of tasks that you have in hand... What you are doing, you have a sticker (Trouble reporters) (Product Owner). ...Because of the program level Kanban board, people are more aware of what happens in other areas, other than their own area. We discuss several areas so people are more aware of the other guys’ work. They understand better the need of others, they reach out to the other guys and bring help and put effort to unblock the situation (Line Manager)’*. Such visibility made the maintenance team robust and able to respond quickly towards blocked tasks in the work flow. The team members try to identify the root causes of the problems and solve it collaboratively as soon as possible. Interviewees expressed their experiences as follows: *‘Nowadays, visibility is different, we react sooner to issues. If there is an issue in one team, then the other teams help more and soon realise that there is a bottleneck in one team, so the other teams need to help solve that. In that sense, I believe that it has helped (Tester)’*. Now the management can efficiently keep track of the up-to-date status of maintenance work. *‘The*

management team can take a look into the team Kanban boards and try to understand more deeply what we are doing and what actions are needed by them (Developer)'. So, visibility also helps on moving things forward when they are stalled (Line Manager)'.

In summary, to handle the lack of visibility challenge, Kanban is more suitable for maintenance work with a short cycle compared to Scrum. According to Kniberg and Skarin [2] the Scrum board is usually visible to whoever is interested, but only the owning Scrum team may edit it to manage commitment for each sprint. On the other hand, the Kanban board is related to one workflow and doesn't need to be owned by one specific team [2]. Changing the way of working from Scrum to Kanban helps to visually demonstrate the individual maintenance task dependences on each other. As a result, team members start to help each other and complete tasks more collaboratively. Additionally, maintenance teams and management see the overall picture of work status at any given time. This supports the findings of [4, 20, 30].

5.2 Solution to fluctuating task priorities

In maintenance work, customers' requests come on a daily basis, and tasks are constantly prioritised based on severity. The maintenance teams can now efficiently handle frequent reprioritisations and constantly changing work using the Kanban board. Similar to the Scrum way of working, Kanban teams work on highest priority tasks; however, when a task is completed, it is immediately delivered, sometimes within a day or even hours. With such continuous reprioritisation and continuous delivery, maintenance teams become highly responsive without overheads, as in the case of Scrum, when trying to fit the tasks into timeboxes.

In summary, Kanban is more appropriate for work (i.e. maintenance) where there is a high degree of variability in priority, whereas Scrum is more suited where work is prioritised in batches and priorities do not shift in less than one sprint duration. Due to the nature of maintenance tasks, teams prefer to work on a continuous delivery model in Kanban, which works better compared to the concept of a sprint delivery in Scrum. In this way, compared to Scrum, Kanban is more suitable for maintenance work. The maintenance teams decided to use five policies to handle fluctuating tasks, which also supports the findings of previous studies [4, 20, 30].

- Do not accept or start work on a task that is not on the Kanban board.
- Discuss task progress and priorities in daily stand-up meetings. A coach and line manager mentioned, *'We gather around the Kanban board every day and we discuss what needs to be done that might switch or change priority from one moment to another'.*

- Tasks should be pulled from the Kanban board according to the highest priority, *'everybody able to pull the highest priority item and work on it, then we agreed that everybody has just one thing ongoing at a time (Coach)'.*
- Once a high-priority task is pulled from the Kanban board, the next one will not be pulled until the first task is completed. A coach pointed out that *'[Kanban] makes the priorities clear. We cannot start another task since we have not finished one of the remaining. There is only one backlog and it has to be in priority order, so the team members do not have to be in the middle of a priority fight'.*
- In the case of extremely urgent or very high-priority critical work, it will go to the express line on the Kanban board, and the whole team will work until its completion. A project manager explained this as follows: *'Sometimes we need to stop ongoing work and focus priorities; it requires more focus from people. In such high-priority work, the tasks are visible in the express or fast line on the Kanban board, which has its own WIP limit'.*

5.3 Solution to unpredictable tasks frequently in process

Previous research shows that two- to four-week or shorter sprints can be used in maintenance work [11, 12]. The case maintenance teams were using two-week sprints, which were not efficacious owing to daily unpredictable incoming tasks, which require quick responses. Such tasks are difficult to handle in a sprint. Further, the isolated and short cyclic nature of the maintenance task made sprints challenging.

To handle unpredictable incoming tasks challenge, maintenance teams start using Kanban. In this way, the teams automatically drop the practice of two-week sprints as prescribed by Scrum. Now, team members start pulling the tasks based on the highest priority from the Kanban board and delivering completed tasks continuously. A product owner expressed this as follows: *'They [team members] are picking their task by themselves. We actually do it quite spontaneously. I have tried to avoid a situation where I dictate that we should do now this and that'.* Additionally, the team also realised that with low WIP limits, teams quickly became idle; with higher WIP limits, the teams had too much task switching and too many tasks going on at a given time with slow delivery. Teams discovered their WIP limits and started following it strictly using the pull technique. A product owner explained, *'WIP limit helps teams to avoid working on too many parallel tasks and are forced to work on those tasks that deliver value to the project. It is policy that once the task is pulled from the Kanban board, it should be finished before taking a new task'.*

5.4 Solution to team sprint commitments

Due to the nature of unpredictable daily-incoming tasks, the maintenance teams are unable to forecast what they would be working on for the next two-week sprint, which makes it difficult for realistic planning and commitments. To handle commitment challenges, first the teams drop timebox iterations, which are not prescribed by Kanban. Second, teams start working in a continuous manner, applying WIP limits and promoting the pull technique to work on highest priority tasks rather than doing unscheduled tasks. The interviewees explained, *'We have a limit for each phase, a limit for the pre-studies, grooming and implementation phase (Tester). We have managed to reduce the queues between different phases by reducing the WIP; we reduce the input and everything in the middle (Product Manager). We agreed about the WIP limits that only a certain amount of tasks at the same time will be ongoing and the others are queuing in the backlog (Coach)'*. The interviewees expressed their experiences about WIP limits as follows: *'force you to just, work on a specific thing. First one and then when that is finished next (Trouble Shooter). We adjust the WIP limit and decrease the features that we compromised on because we start to understand the team's velocity (Developer)'*. A coach expressed that *'the big value of Kanban is the work in progress control. One experienced guy saying that for once he has the peace to just focus on doing one thing at a time and really focus on it, and he said how great it feels (Coach)'*. Such reported WIP limit principle benefits are also supported by previous studies [4, 30].

5.5 Solution to work synchronisation and changing people

Maintenance team tasks are a mix of enhancements and corrections from different customers, which reduce the sense of focus. Furthermore, maintenance team members mostly work alone on tasks and often have sole responsibility for their tasks. With Kanban implementation, each team member's tasks became visible on the Kanban board, which made it easier to synchronise work with management and other team members. The managers became more aware about each team and its individual members' work. The interviewees explained, *'As coach to me, it [Kanban] provides a structured way of work flowing to people in the teams (Coach). Visibility of our work and the synchronisation between the R&D teams, management teams and even inside the different R&D teams improve a lot. The team spirit [improves] because we have a target. That is, let's stop starting and start finishing (Developer)'*.

Furthermore, Kanban helps management in resource allocation and changing people across teams when it is needed. For example, when a task is blocked or is not

moving forward owing to required expert knowledge, managers easily see it and try to guide the team members to focus on the blocked work collaboratively. The management moves expert people from one team to another to sort out the problems and share the knowledge and expertise of senior team members. In such cases, the team members work as in pair programming. An interviewee explained, *'People are more aware of what happens on other than their own area. Because on a program level Kanban board, we discuss several areas that we have to deal with so people are more aware of the other guys' work, and that brings the benefit that they understand better the need of other guys and bring some help to unblock the situation (Coach and Line Manager). It is easier now to move people to another team for some time (Project Manager)'*. Such experiences are also supported by [4, 20, 30].

5.6 Solution to lack of communication and collaboration

Maintenance teams work on fixing bugs or making changes, such as modifications and enhancements that are requested by the customer. Such requests are varied in nature, technical depth and severity and require frequent communication among maintenance teams to understand the customer's view and prioritise tasks according to business values. To enhance teamwork, the Kanban way of working is supplemented with daily stand-up meetings and retrospectives. In our studied maintenance teams, it was decided that each team would have daily stand-up meetings and conduct meetings twice a week with other maintenance teams and management. In these meetings, the updated status of completed and ongoing tasks and impediments were discussed.

To avoid team member isolation and make their work more visible, each team had its own Kanban board. Additionally, one common Kanban board was also designed for the whole maintenance unit on which each team displayed their work status. Such practices help to communicate work status inside and outside the teams. A line manager explained, *'we keep the Kanban rolling. A lot of stuff is happening every day, a lot of information from all the projects that we need to share. This is one way of communicating to the whole project'*.

Each team's Kanban board makes the discussion more open and enables collaborative work to solve problems when they occur. For instance, if a maintenance task is visible on the Kanban board and a team member is already familiar with it, he or she starts to explain how to solve it. An interviewee expressed these experiences as follows: *'...earlier [before Kanban use] you might not even know what others are doing. Now, you can see it on the Kanban board, and you discuss it in the dailies. I would say that there is a lot of collaboration (Coach). We do support each other with*

ideas and advice (Tester). ...it was very easy, the communication and collaboration is somehow increased and goes in a positive direction. We all [work towards] to the same objective, to the same target. And that's, the great benefit from Kanban'.

Experts or more senior team members help the team members whenever they see a blocking task on the Kanban board. An interviewee expressed, *'The whole team can scan what you're doing, and if they recognise a problem that they had before or have experience with, they can jump in and offer some advice or assistance (Tester)'*. Additionally, such team work helps to promote an environment in which senior and/or domain experts share their knowledge with other members. A coach explained, *'People have learned from each other's knowledge areas or they have learned new things that enables solving cases faster and more effectively'*. Such experiences are aligned with the findings of previous studies [4, 20, 30].

6. Limitations and validity discussion

Threats to the validity of the outcome of the study are important to consider during the design of the study as this allows actions to be taken to mitigate them. We consider four perspectives of validity threats: construct validity, internal validity, external validity and reliability [13].

Construct validity is concerned with obtaining the right measures for the concept being studied. One threat was the selection of interviewees to obtain the appropriate sample for answering the research questions. Therefore, experienced interviewees within the companies were selected, keeping the following aspects in mind: knowledge of Kanban, software development process, and experience of using Kanban in software development and maintenance areas. Construct validity was also threatened if the interview questions were misunderstood or misinterpreted. Therefore, at the beginning of each interview session, interviewees were introduced to key concepts used in the study to avoid misinterpretation. Additionally, pre-test interviews were conducted. Internal validity focuses on how to establish a causal relationship and is mainly used for explanatory and causal studies. As this study was of an exploratory nature, internal validity was not considered. External validity is the ability to generalise the findings from a specific context of the study to other contexts. The findings of our study provide insights into Kanban usage in software maintenance and release teams from two large software companies from the N4S program. The findings should be considered with caution; there is always the risk that the outcome of the study is affected by the interpretation of the researcher. In the initial phase of this study, one researcher created the study protocol and the remaining researchers reviewed the protocol. The threat to reliability is concerned with repetition or replication; in particular,

the same result should be obtained upon repeating the study with the same settings. There is always a risk that the outcome of the study is affected by the interpretation of the researcher. To mitigate this threat, the study was designed so that data were collected through interviews from two different companies in which different roles of the participants were taken into consideration. Additionally, in each phase of the research process, at least two researchers were involved to reduce the risk of bias.

7. Lessons learned by teams

Kanban is mainly used in software development; however, in light of our study, its practices appear advantageous in maintenance as well [5, 15, 21]. The case teams experienced benefits, including increased visibility of tasks, improvement of team morale and better communication and knowledge sharing. Based on our study, we extracted the following lessons learned for maintainers adopting the Kanban:

- Maintenance teams are handling daily customer requests that require constant prioritisation. For efficient prioritisation, it is essential to have clear visibility of all customer requests. In this regard, the Kanban board helps in visualising maintenance team work. For example, if a team member has difficulties or trouble with a specific maintenance task that is blocked and results in a delay in the completion of a customer request, the other team members or specialists spot it on the Kanban board and jump in to sort out the problem. In our study, team members sometimes used the pair programming technique. As a result, work was completed efficiently and led team members to work on diverse tasks and broaden their working domain area.
- Using a WIP limit forces the maintenance team members to focus on and control their work. This practice helps teams in defining the definition of 'Done', which is enhancement for removing the partially done work throughout the process.
- Promote the pull approach. The proactive role of team leaders is essential in Kanban boards. The team leader and team members prioritise the maintenance tasks daily based on severity. This practice enables team members to autonomously 'pull' a task from the Kanban board as their capacity frees up and removes the option of managers pushing tasks to individuals.
- Most maintenance tasks are short cyclic and isolated. These tasks are visualised on the Kanban board in columns. In daily maintenance work, sometimes very urgent customer requests enter the Kanban board. To capture the attention of the whole team and complete such an urgent task, an express line is needed. The express line signals the

team of tasks that need urgent attention and need to be delivered quickly.

- Whether to use sprints in maintenance work depends on the maintenance team. Maintenance work is very unpredictable, and tasks do not come at a steady pace. If maintenance tasks can wait one week to be completed, then sprints may work; however, this will not be an optimal solution. The alternative is Kanban, which focuses on continuous delivery. All the incoming tasks are on the Kanban board and can have priorities based on urgency and value. Kanban teams can easily see these tasks on the Kanban board express line. Kanban makes the team more proactive when high-priority work comes in, rather than waiting for an iteration to complete.
- Keep daily stand-up meetings regular. This provides up-to-date information about work to all stakeholders. As the maintenance work is unpredictable and tasks are prioritised on a daily basis, frequent reprioritisation of tasks during a day makes the team frustrated. The daily stand-up meetings help in prioritisation every morning, which helps to focus on daily work.

8. Conclusion

Software maintenance takes up a considerably large percentage of the software cost. It is important to adopt a method or process that is efficient and easy to follow for the maintenance team. During the last decade, Agile methods have been adopted in software development and are also beginning to be used by software maintenance teams. However, relatively little research has focused on these methods for maintenance. In this study, we investigated what challenges led two experienced Scrum software maintenance teams in large companies to start using Kanban and how they tackled those challenges using Kanban.

This study is not to disrupt Scrum but rather to look for challenges faced by maintenance teams while practicing Scrum in their work. We conducted a multiple-case study on maintenance teams from two large Finnish software companies. We found five major challenges in these two maintenance teams when using Scrum. We described solutions that the maintenance team developed to improve their work practice. Such improvement could be achieved by multiple ways, but in case teams Kanban makes more sense. For example, tasks are visualised on the Scrum board for each sprint, while Kanban visualises tasks that can be pulled at any time while considering WIP limits; this restricts the allowed number of tasks in every workflow state.

The results indicate that Kanban is more appropriate for work where there is a high degree of variability in priority. It can also be argued that Scrum is more

appropriate for work where the tasks can be prioritised in batches and where priorities do not shift on a frequent basis. Currently, Kanban is being increasingly adopted to complement Scrum and other Agile methods in software processes [5]. Our findings show that Kanban offers various benefits to maintenance work, such as bringing visibility to maintenance tasks, protecting teams from over-commitment, helping in task prioritisation, synchronising work with other teams and management, easily changing people between teams, improving communication and promoting collaboration in teamwork. By using Kanban, team action towards urgent work is more spontaneous, and work can be pulled according to high priorities. Additionally, the WIP limits improve maintenance team throughput and efficiency. These results align with the reported benefits of Kanban in the software development literature. It is now evident that Kanban is an evolutionary method that can be optimised to a working situation in which tasks are frequently changing, for example, maintenance work.

In the future, we plan to augment this study by performing additional case studies with other teams on Kanban in the software development and maintenance domains. To obtain a better understanding, we are planning to collect more qualitative and quantitative data to analyse how Kanban helps software development and maintenance teams in their work.

9. Acknowledgements

The authors thank the companies and their employees for participating in this research. This research has been carried out in the Digile Need for Speed program, and it has been partially funded by Tekes (the Finnish Funding Agency for Technology and Innovation) and a Nokia foundation scholarship.

10. References

This is a draft paper, final version can be found in proceedings of IEEE 49th Hawaii International Conference on System Sciences (HICSS)

This work can be cited as:

Ahmad, M. O., Kuvaja, P., Markkula, J. & Oivo, M. (2016). Transition of software maintenance teams from Scrum to Kanban. IEEE 49th Hawaii International Conference on System Sciences (HICSS).